

Decorator

1. Explicación del patrón

Decorator es un patrón estructural que permite añadir funcionalidades adicionales a un objeto de forma dinámica, sin modificar su interfaz.

La idea principal es envolver un objeto dentro de otro objeto decorador.

El decorador tiene la misma interfaz que el objeto original, por lo que el cliente puede usarlo de la misma manera. Sin embargo, el decorador añade comportamiento antes o después de delegar la operación al objeto envuelto.

De esta forma, se pueden combinar varias funcionalidades envolviendo un objeto con varios decoradores.

El patrón Decorator es una alternativa al uso de herencia para extender el comportamiento de una clase.

En lugar de crear muchas subclases para cubrir todas las combinaciones posibles de funcionalidades, se crean decoradores independientes que pueden añadirse o quitarse según se necesiten.

2. Cuándo se utiliza

Se utiliza el patrón Decorator cuando:

- Se quieren añadir funcionalidades a un objeto sin modificar su interfaz.
- El cliente no debe verse afectado por las funcionalidades añadidas.
- Se necesitan añadir funcionalidades dinámicamente durante la ejecución.
- Se necesitan eliminar funcionalidades dinámicamente.
- El uso de herencia para extender objetos no es práctico.
- La jerarquía de clases ya es compleja.
- Se quieren combinar varias funcionalidades opcionales.
- Se quiere evitar crear una subclase por cada combinación posible de comportamientos.
- Se quiere extender un objeto concreto sin afectar al resto de objetos de la misma clase.

3. Cómo aplicarlo

Para aplicar Decorator se suelen definir estos participantes:

Componente

Es la interfaz común que comparten el objeto original y todos los decoradores.

Define las operaciones que el cliente puede usar.

Ejemplo conceptual:

- `operacion()`

- `getDescripcion()`
- `getPrecio()`

En el ejemplo de coches:

- `IConfiguradorCoche`

En el ejemplo de pizzas:

- `Pizza`

Componente concreto

Es el objeto base que se quiere decorar.

Contiene el comportamiento original.

Responsabilidades:

- Implementar la interfaz común.
- Proporcionar el comportamiento básico.
- Poder ser envuelto por uno o varios decoradores.

Ejemplos del material:

- `ComponenteConcreto`
- `CocheBase`
- `PizzaBasica`

Decorador abstracto

Es una clase abstracta que también implementa la interfaz del componente.

Además, mantiene una referencia a un objeto del mismo tipo que la interfaz común.

Responsabilidades:

- Guardar el objeto envuelto.
- Implementar la misma interfaz que el componente.
- Delegar las llamadas al objeto envuelto.
- Servir como base para los decoradores concretos.

Ejemplos del material:

- `Decorador`
- `DecoradorCoche`
- `IngredienteDecorator`

Decorador concreto

Es una clase que añade una funcionalidad específica.

Responsabilidades:

- Extender el decorador abstracto.
- Añadir comportamiento antes o después de llamar al objeto envuelto.
- Mantener la misma interfaz que el componente original.

Ejemplos del material:

- DecoradorConcretoA
- DecoradorConcretoB
- Decorador_AireAcondicionado
- DecoradorNavegadorGPS
- DecoradorTechoSolar
- QuesoExtra
- Pepperoni
- Champiñones

Cliente

Es quien utiliza el componente.

El cliente trabaja con la interfaz común y no necesita saber si está usando:

- Un componente simple.
- Un componente decorado.
- Un componente con varios decoradores.

4. Forma práctica de aplicarlo

Pasos:

1. Identificar una clase a la que se le quieren añadir funcionalidades opcionales.
2. Crear una interfaz común para el objeto base y sus decoradores.
3. Hacer que la clase base implemente esa interfaz.
4. Crear una clase decoradora abstracta que también implemente la interfaz.
5. Añadir en el decorador abstracto una referencia al componente que se va a envolver.
6. Hacer que el decorador abstracto delegue las operaciones al componente envuelto.
7. Crear decoradores concretos para cada funcionalidad adicional.
8. En cada decorador concreto, añadir el comportamiento extra.
9. Envolver el objeto base con uno o varios decoradores según las necesidades.
10. Hacer que el cliente use siempre la interfaz común.

5. Idea clave

El objeto original no se modifica.

Se envuelve con decoradores que añaden nuevas responsabilidades.

Todos los decoradores mantienen la misma interfaz, por lo que el cliente puede usar el objeto decorado como si fuera el objeto original.

6. Funcionamiento interno

El patrón funciona mediante composición.

Un decorador contiene una referencia a un componente.

Ese componente puede ser:

- El objeto base.
- Otro decorador.

Esto permite formar una cadena o pila de decoradores.

Por ejemplo:

```
DecoradorB -> DecoradorA -> ComponenteConcreto
```

Cuando el cliente llama a una operación sobre `DecoradorB`, ocurre lo siguiente:

1. `DecoradorB` ejecuta su comportamiento.
2. `DecoradorB` delega en `DecoradorA`.
3. `DecoradorA` ejecuta su comportamiento.
4. `DecoradorA` delega en `ComponenteConcreto`.
5. `ComponenteConcreto` ejecuta el comportamiento base.
6. La respuesta vuelve hacia atrás atravesando los decoradores.

Así se obtiene un comportamiento combinado.

7. Ejemplos del material

Ejemplo general

En el diagrama general aparece una interfaz:

- `I_ComponenteAbstracto`

Con operaciones como:

- `operacion()`
- `getNumeroDecoradores()`

A partir de ella se definen:

- `ComponenteConcreto`
- `Decorador`
- `DecoradorConcretoA`
- `DecoradorConcretoB`

El decorador abstracto tiene una referencia a `I_ComponenteAbstracto`.

Esto permite que el decorador pueda envolver tanto al componente concreto como a otro decorador concreto.

Ejemplo de composición en tiempo de ejecución:

```
I_ComponenteAbstracto c = new DecoradorConcretoB(new DecoradorConcretoA(new ComponenteConcreto()))
```

La estructura resultante sería:

- DecoradorConcretoB
 - DecoradorConcretoA
 - ComponenteConcreto

Ejemplo de configurador de coche

En el material aparece un ejemplo donde se configura un coche añadiendo extras.

Interfaz común:

- IConfiguradorCoche

Operaciones:

- getDescripcion()
- getCoste()

Componente concreto:

- CocheBase

Decorador abstracto:

- DecoradorCoche

Decoradores concretos:

- Decorador_AireAcondicionado
- DecoradorNavegadorGPS
- DecoradorTechoSolar

Cada decorador añade un extra al coche.

Por ejemplo, un coche puede tener aire acondicionado, navegador GPS y techo solar.

La composición puede hacerse en tiempo de ejecución:

```
IConfiguradorCoche coche = new DecoradorTechoSolar(new DecoradorNavegadorGPS(new Decorador_AireAcondicionado(new CocheBase())))
```

La estructura resultante sería:

- DecoradorTechoSolar
 - DecoradorNavegadorGPS

- Decorador_AireAcondicionado
- CocheBase

El cliente sigue usando los métodos `getDescripcion()` y `getCoste()` , sin importar cuántos extras tenga el coche.

Ejemplo de pizza

En el material también aparece un ejemplo de pizzas.

Interfaz común:

- Pizza

Operaciones:

- `getDescripcion()`
- `getPrecio()`

Componente concreto:

- PizzaBasica

Decorador abstracto:

- IngredienteDecorator

Decoradores concretos:

- QuesoExtra
- Pepperoni
- Champiñones

Cada ingrediente funciona como un decorador.

Una pizza básica puede envolverse con varios ingredientes adicionales.

Por ejemplo:

```
Pizza pizza = new Champiñones(new Pepperoni(new QuesoExtra(new PizzaBasica())))
```

El cliente puede pedir:

- `pizza.getDescripcion()`
- `pizza.getPrecio()`

La descripción y el precio final se calculan acumulando el comportamiento de todos los decoradores.

8. Ejemplo conceptual de uso

Sin Decorator, si se quisieran muchas combinaciones de extras, habría que crear muchas subclases:

- CocheConGPS

- CocheConTechoSolar
- CocheConAireAcondicionado
- CocheConGPSYAire
- CocheConGPSTecho
- CocheConTodo

Con Decorator, se crean decoradores independientes:

- GPS
- TechoSolar
- AireAcondicionado

Y se combinan dinámicamente:

```
new TechoSolar(new GPS(new AireAcondicionado(new CocheBase())))
```

Así se evita una explosión de subclases.

9. Ventajas prácticas

- Permite añadir funcionalidades sin modificar la clase original.
- Permite añadir o quitar responsabilidades durante la ejecución.
- Evita crear muchas subclases para distintas combinaciones.
- Mantiene la misma interfaz para el cliente.
- Permite combinar varios decoradores.
- Separa cada funcionalidad adicional en una clase independiente.
- Facilita extender objetos concretos sin afectar a otros objetos.
- Es más flexible que la herencia para añadir comportamiento opcional.

10. Cuidado

Decorator puede aumentar el número de clases del sistema.

También puede hacer que el flujo sea más difícil de seguir si se encadenan demasiados decoradores.

El orden de los decoradores puede ser importante.

Por ejemplo:

```
DecoradorA -> DecoradorB -> Componente
```

puede no comportarse igual que:

```
DecoradorB -> DecoradorA -> Componente
```

Además, eliminar un decorador concreto de una cadena de decoradores puede ser difícil si la pila ya está construida.

11. Diferencia práctica con otros patrones estructurales

Diferencia con Adapter

Adapter cambia la interfaz de un objeto para que sea compatible con lo que espera el cliente.

Decorator mantiene la misma interfaz y añade comportamiento.

Por tanto:

- Adapter sirve para hacer compatibles interfaces distintas.
- Decorator sirve para añadir responsabilidades a un objeto.

Diferencia con Composite

Composite representa estructuras en árbol con objetos simples y compuestos.

Decorator normalmente envuelve un único componente para añadirle comportamiento.

Por tanto:

- Composite se centra en componer objetos en jerarquías.
- Decorator se centra en añadir funcionalidades mediante envoltorios.

Diferencia con Proxy

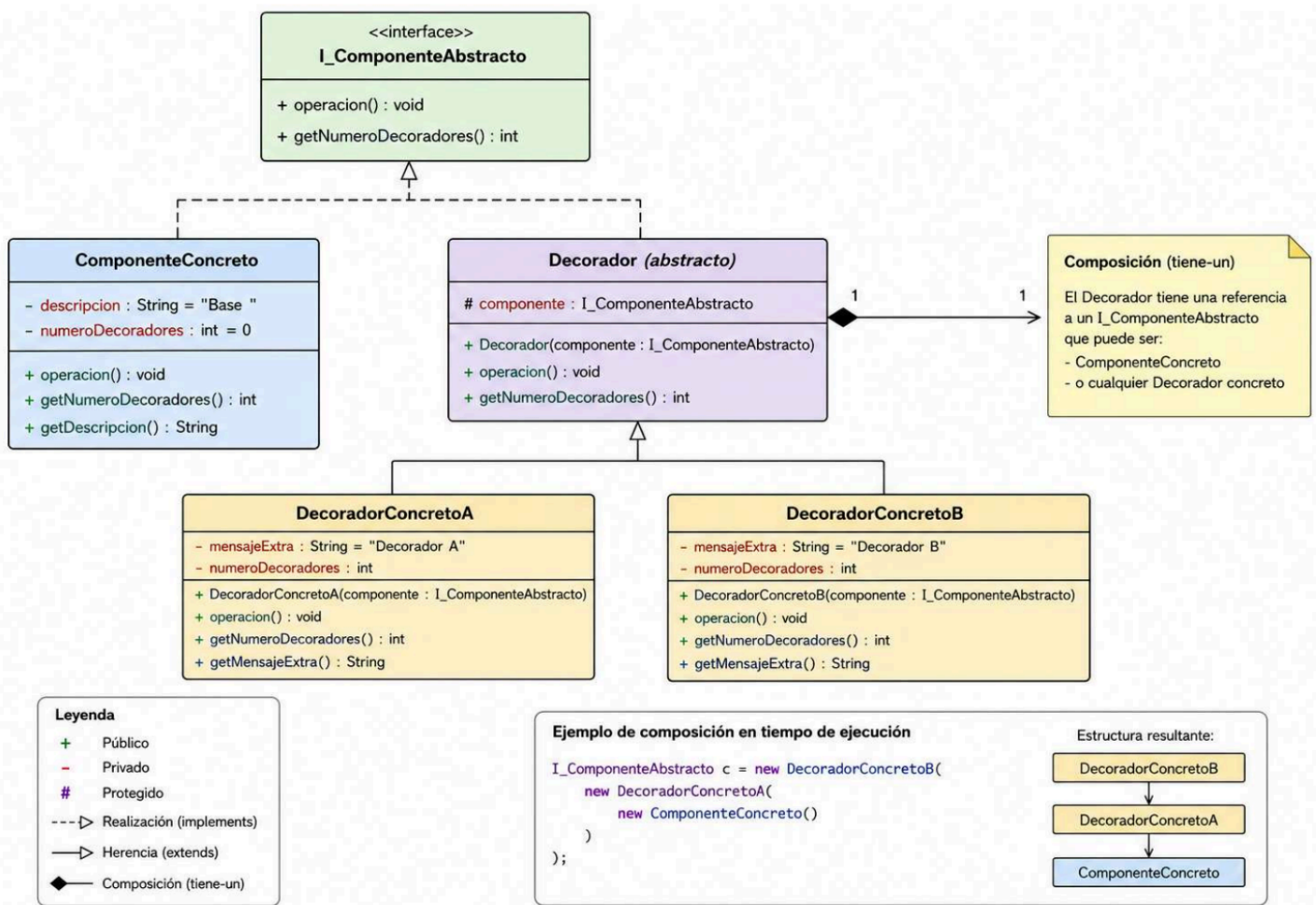
Proxy controla el acceso a un objeto.

Decorator añade comportamiento extra a un objeto.

Por tanto:

- Proxy se usa para controlar acceso, carga, permisos o intermediación.
- Decorator se usa para extender comportamiento.

12. Plantilla



13. Ejemplo

